SECUREVAULT – AN ENTERPRISE CREDENTIAL MANAGEMENT PLATFORM

A PROJECT REPORT

Submitted by

ADITYA KAMATH V(310624148601)

in partial fulfillment for the award of the Degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

2311CSE503R - FULL STACK DEVELOPMENT



EASWARI ENGINEERING COLLEGE,

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled **"SECUREVAULT – AN ENTERPRISE CREDENTIAL MANAGEMENT PLATFORM"** is the Bonafide work of **"ADITYA KAMATH V(310624148601)",** who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT
Dr. Justin Dhas Y
Department CSE(AIML)
Easwari Engineering College

SUPERVISOR
Ms. Divyanjali
Easwari Engineering College

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "**SecureVault – An Enterprise Credential Management Platform**" submitted by **Mr. ADITYA KAMATH V**(Reg. No. 310624148601) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** was conducted on _____ (Date) in the Department of **Computer Science and Engineering (AI & ML)**, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Name :

Designation:

Address :

External Examiner

Name :

Designation:

Address :

Date:

ABSTRACT

SecureVault is an organisation-level enterprise credential management platform built on the MERN (MongoDB, Express.js, React.js, Node.js) technology stack. The project addresses two widespread security weaknesses in modern organisations: password reuse with weak entropy, and uncontrolled credential sprawl, where server passwords, API keys, database credentials and service accounts are shared over unsecured channels such as chat tools, spreadsheets and plaintext notes.

SecureVault resolves these problems by combining AES-256-GCM authenticated encryption for stored credentials, bcrypt-hashed master passwords, JWT-based stateless authentication, role-based access control (RBAC), fine-grained credential sharing with cascading revocation, zxcvbn-based password health auditing, and an immutable audit trail in one unified interface. A zero-knowledge administrative boundary ensures that administrators see only aggregate security statistics and can never decrypt employee credentials.

The front end is a React 18 single page application built with Vite, and the back end is a modular MVC-style Express.js REST API backed by four Mongoose models (User, Credential, SharedAccess and AuditLog). An embedded in-memory MongoDB fallback with an automated seeder allows the complete system to be demonstrated without any manual database setup. The project demonstrates full-stack proficiency, defensive cryptographic engineering and practical applicability to real-world enterprise credential governance.

Keywords: Credential Management, AES-256-GCM, bcrypt, JWT, Role-Based Access Control, Zero-Knowledge, Audit Trail, MERN Stack.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal **Dr. P. Deiva Sundari** and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department **Dr. Justin Dhas Y**, Department of Computer Science and Engineering (AI & ML), for the continuous encouragement and support.

We are extremely thankful to our Project Guide, **Ms. Divyanjali** Department of Computer Science and Engineering (AI & ML), for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Computer Science and Engineering (AI & ML) for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
1	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	1
	1.3 OBJECTIVE	1
	1.4 SCOPE OF THE PROJECT	2
	1.5 MOTIVATION	2
2	**RELATED WORK**	**3**
	2.1 LITERATURE REVIEW	3
	2.2 EXISTING SYSTEM	3
	2.3 PROPOSED SYSTEM	4
3	**SYSTEM ANALYSIS**	**5**
	3.1 SYSTEM REQUIREMENTS	5
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	6
4	**SYSTEM DESIGN**	**7**
	4.1 SYSTEM ARCHITECTURE	7
	4.2 MODULE DESCRIPTION	7
	4.3 DATABASE DESIGN	9
	4.4 CLASS DIAGRAM	10
	4.5 SEQUENCE DIAGRAM	10
	4.6 KEY DESIGN DECISIONS	11
5	**IMPLEMENTATION**	**13**
	5.1 IMPLEMENTATION OVERVIEW	13
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	14
6	**CONCLUSION & FUTURE WORK**	**18**
	6.1 CONCLUSION	18
	6.2 FUTURE ENHANCEMENTS	18
	APPENDIX	**19**
	SCREENSHOTS OF THE PROJECT	**21**
	REFERENCES	**23**

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

SecureVault is an organisation-level enterprise credential management platform built on the modern MERN (MongoDB, Express.js, React.js, Node.js) technology stack. It provides employees with a secure, central vault in which they can store, search, generate, share and audit the credentials they use every day, while giving administrators a governance view of the organisation's overall password security posture.

The platform combines AES-256-GCM cryptographic encryption, role-based access control (RBAC), fine-grained credential sharing with cascading revocation, automated entropy health auditing, and an immutable audit trail into a unified, zero-knowledge interface.

1.2 PROBLEM DEFINITION

The project is designed to resolve two rampant security vulnerabilities that affect modern businesses:

1. **Password Reuse & Weak Entropy:** Employees frequently reuse identical, predictable passwords across company services, vendor dashboards and personal portals. An attacker who compromises a single trivial service can therefore compromise mission-critical company infrastructure.
2. **Uncontrolled Credential Sprawl:** Internal teams frequently share sensitive server passwords, API keys, database credentials and service accounts using unsecured channels such as Slack, Microsoft Teams, spreadsheets or plaintext notes, resulting in unmonitored access and zero accountability.

1.3 OBJECTIVE

The main objective of the project is to design and implement a secure, role-aware credential management platform that eliminates unsafe credential practices inside an organisation. The specific objectives are:

- To store every credential only as AES-256-GCM ciphertext with a unique initialization vector and an integrity-protecting authentication tag.
- To protect user master passwords with salted, adaptive bcrypt hashing (12 salt rounds) so that they can never be retrieved.
- To enforce strict separation between employee and administrator capabilities using JWT authentication and RBAC guards on both the client and the server.
- To allow granular sharing of individual credentials (view-only or edit) with instant revocation, including automatic cascade revocation when an account is deactivated.
- To measure password strength with realistic pattern-based entropy analysis and to report weak, reused and unrotated (90+ days) credentials at the personal and organisational level.
- To record every sensitive action in an immutable audit trail that supports searching, filtering and CSV export for compliance audits.
- To enforce a zero-knowledge administrative boundary in which administrators see only aggregate statistics and never plaintext credentials.

1.4 SCOPE OF THE PROJECT

The scope of SecureVault covers a complete full-stack credential governance system for an organisation with two user roles, **employee** and **admin**.

- **Employee scope:** registration with a real-time password strength meter, login, a personal credential vault with search and filters, a built-in high-entropy password generator, on-demand decryption and clipboard copy, sharing with named colleagues, and a personal password health report.
- **Administrator scope:** an organisation-wide governance dashboard, employee directory with activate/deactivate control, organisation-wide password strength distribution (without exposing plaintext), and a searchable audit log with CSV export.
- **Platform scope:** REST API with validation and centralised error handling, four normalised database collections, a zero-setup embedded database fallback with automated seeding, and a developer-friendly workspace with one-command start-up.

1.5 MOTIVATION

A single reused or weak password can expose an entire organisation, and the informal sharing of secrets over chat tools and spreadsheets leaves no record of who accessed what. These problems are common, easily preventable and costly when exploited. This motivated the development of a platform that makes the secure behaviour the easy behaviour: strong passwords are generated automatically, credentials are shared with controlled permissions instead of being pasted into messages, and every access is accountable through the audit trail.

The project also provides a practical opportunity to apply core security engineering concepts, namely authenticated encryption, adaptive hashing, stateless authentication, access control and auditing, in a complete MERN application.

CHAPTER 2

RELATED WORK

2.1 LITERATURE REVIEW

SecureVault builds on a set of well-established security techniques. The key ideas from the literature that informed its design are summarised below.

- **Adaptive password hashing.** Provos and Mazières introduced bcrypt as a future-adaptable password scheme whose cost factor can be increased as hardware improves. Salting and a configurable work factor make large-scale brute-force and dictionary attacks expensive even when a password database is leaked. SecureVault applies bcrypt with 12 salt rounds to master passwords.
- **Authenticated encryption.** NIST Special Publication 800-38D specifies Galois/Counter Mode (GCM), which provides both confidentiality and data authenticity through an authentication tag. Unlike confidentiality-only modes such as CBC, tampering with the ciphertext or IV is detected at decryption time. SecureVault uses AES-256-GCM for all stored credentials.
- **Realistic password strength estimation.** Wheeler's zxcvbn estimator evaluates passwords against dictionaries, names, repeated sequences, l33t substitutions and keyboard patterns, and is a more accurate measure of cracking resistance than simple composition rules such as "eight characters and one symbol". SecureVault uses zxcvbn for its real-time meter and for health audits.
- **Stateless token authentication.** JSON Web Tokens (RFC 7519) carry signed claims that a server can verify without maintaining session state, which supports horizontal scaling. SecureVault issues signed JWTs containing the user ID and role with an 8-hour expiry.
- **Secure password storage guidance.** The OWASP Password Storage Cheat Sheet recommends salted adaptive hashing for authentication secrets and separates this from the reversible encryption needed for secrets that must be read back later. This is the basis of SecureVault's dual use of hashing and encryption.

2.2 EXISTING SYSTEM

In many organisations credentials are still managed informally. Teams share server passwords, API keys, database credentials and service accounts through Slack, Microsoft Teams, spreadsheets or plaintext notes. Employees reuse identical and predictable passwords across company services, vendor dashboards and personal portals.

The limitations of this existing approach are:

- Credentials are stored and transmitted in plaintext on unsecured channels.
- There is no access control: anyone who sees the message or file can use the credential.
- Access is unmonitored and there is no accountability or audit history.
- When an employee leaves, there is no way to revoke the credentials that were shared with or by them.
- Weak and reused passwords go undetected because there is no strength measurement.

2.3 PROPOSED SYSTEM

SecureVault replaces informal credential handling with a single encrypted, role-aware platform. Its main characteristics are:

- **Encrypted storage:** every credential is encrypted with AES-256-GCM using a unique random 16-byte IV, and the 16-byte authentication tag is validated on decryption.
- **Strong identity protection:** master passwords are hashed with bcrypt (12 rounds) and are never stored in plaintext or returned in responses; JWT tokens expire after 8 hours.
- **Controlled sharing:** credentials can be shared with named colleagues as view-only or edit and revoked instantly; deactivating an employee automatically revokes all of their shared access.
- **Health auditing:** zxcvbn-based scoring identifies weak, reused and 90-day-old credentials for each employee and across the organisation.

- **Zero-knowledge administration:** administrators view only aggregate metrics and cannot decrypt employee credentials.
- **Accountability:** all sensitive actions are written to an append-only audit log with IP address and timestamp, filterable and exportable as CSV.

CHAPTER 3

SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

Functional Requirements

- User registration and login using a master password, with a real-time password strength meter at registration.
- Create, read, update and delete personal credentials, with search by title/username and filtering by tag and strength.
- Built-in password generator with selectable character sets and a length of 8 to 64 characters.
- On-demand decryption (“Reveal”) of a credential and one-click clipboard copy.
- Share a credential with a colleague as view-only or edit, and revoke access at any time.
- Personal password health report showing overall vault security percentage, weak passwords, reused passwords across distinct domains, and credentials not rotated for 90+ days.
- Administrator dashboard, employee directory with active/inactive toggle, organisation-wide password health distribution and audit log with action filtering and CSV export.

Non-Functional Requirements

- **Security:** AES-256-GCM encryption at rest, bcrypt hashing, signed expiring JWTs, strict input validation and no secrets in source code.
- **Confidentiality:** plaintext credentials are never stored or logged, and administrators cannot decrypt employee credentials.
- **Accountability:** immutable, append-only audit trail with IP address and timestamp.
- **Scalability:** stateless authentication allows multiple backend instances without a shared session store.

- **Usability:** responsive single page interface, modal workflows, demo accounts and visual strength feedback.
- **Maintainability and portability:** modular MVC structure and a zero-setup in-memory database fallback so the system can be run immediately by evaluators.
- **Reliability of error handling:** a centralised error handler returns standardised JSON errors without leaking internal stack traces.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

Software / Tool	Version	Purpose
Node.js	v24	Back-end runtime environment
Express.js	v4.21	Back-end web framework
React.js	v18.3	Front-end framework
Vite	v6.1	Front-end build tool and dev server
MongoDB & Mongoose	v8.9 (Mongoose)	Database and object-document mapper
mongodb-memory-server	v11.3	Embedded in-memory database fallback
bcrypt	v5.1	Master password hashing
jsonwebtoken	v9.0	JWT issuance and verification
zxcvbn	v4.4	Password strength estimation
express-validator	v7.2	Request validation and sanitisation
axios	v1.7	HTTP client for the front end
react-router-dom	v6.28	Client-side routing
lucide-react	v0.475	Icon library
dotenv / cors	v16.4 / v2.8	Environment configuration / CORS
nodemon / concurrently	v3.1 / v10.0	Development tooling
Postman, VS Code, Git	–	API testing, IDE and version control

Hardware Requirements

Component	Requirement
Processor	Dual-core 2.0 GHz or higher
RAM	4 GB minimum (8 GB recommended)
Storage	1 GB of free disk space
Display / Network	Standard display; internet access for installing dependencies
Browser	Latest version of a modern browser (Chrome, Edge or Firefox)

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

SecureVault follows a three-tier architecture. A React 18 single page application (the presentation tier) communicates over HTTP/JSON with an Express.js REST API (the application tier), which stores data in MongoDB through Mongoose (the data tier). During development the Vite dev server proxies /api requests to the backend on http://localhost:5000, which avoids CORS issues.

Every protected request passes through a sequential middleware pipeline on the server: CORS whitelist, JSON body parsing, JWT authentication, RBAC authorisation, express-validator sanitisation, controller business logic, and finally the centralised error handler. Sensitive actions are recorded by the audit logger. Credentials are encrypted and decrypted only in the cryptographic engine using the secret key held in the server environment.

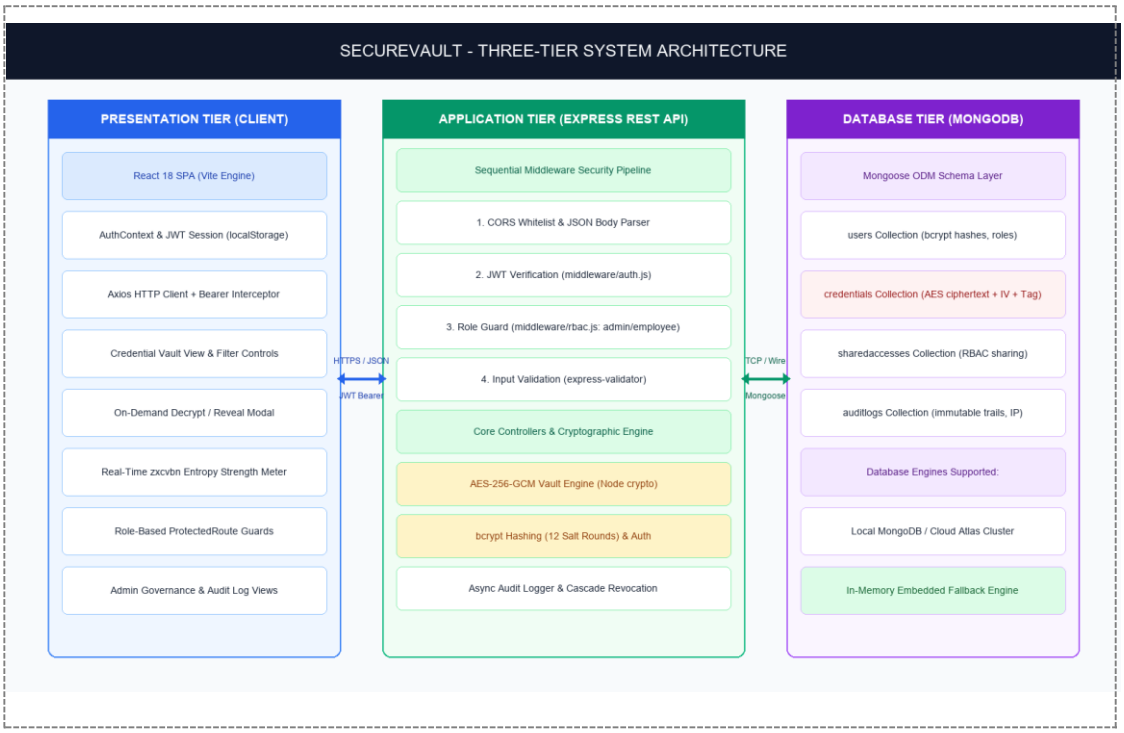


Figure 4.1 System Architecture of SecureVault

4.2 MODULE DESCRIPTION

4.2.1 Frontend Modules (React 18 + Vite)

- **Authentication Screens:** login screen with one-click “Demo Account” quick-fill buttons, and a registration screen with an integrated real-time zxcvbn strength meter (0–4 score bars and colour-coded labels).
- **Route Protection & RBAC:** React Router v6 routing with a custom ProtectedRoute component that verifies JWT existence, expiry and user role, redirecting unauthenticated users to /login.
- **Global Session Management:** a centralised AuthContext that persists the user profile and signed JWT in localStorage and manages login, registration and logout states.
- **Axios HTTP Interceptor:** a central Axios instance (services/api.js) that injects the Bearer token into every request and handles 401 responses by gracefully redirecting expired sessions.
- **Personal Credentials Vault:** responsive table with real-time title/username search, tag and strength filtering, and badges that distinguish owned from shared credentials.
- **Credential Add/Edit Modal:** modal with an integrated password generator (uppercase, lowercase, numbers, symbols, length slider from 8 to 64) and a strength check before submission.
- **On-Demand Decrypt & Copy:** passwords stay masked by default; “Reveal” triggers a decryption request to the backend, with one-click clipboard copy and visual confirmation.
- **Colleague Sharing Modal:** share an individual credential with a named colleague as view-only or edit, with one-click instant revocation.
- **Personal Password Health:** overall vault security percentage, weak passwords, passwords reused across distinct domains, and credentials unrotated for 90+ days.
- **Admin Governance Dashboard:** organisation-wide credential counts, average strength, high-risk counts (weak, reused, expired) and a live recent-activity audit stream.

- **Admin User Management:** employee directory with active/inactive toggle; deactivation cascades revocation of all shared credentials.
- **Organisation Password Health:** strength distribution across Very Weak, Weak, Fair, Strong and Very Strong without exposing plaintext.
- **Admin Audit Log & CSV Export:** searchable audit table with filters for login, create, view, edit, share, revoke and delete, and one-click CSV export.

4.2.2 Backend Modules (Express.js REST API)

- **Server Bootstrap (server.js):** CORS whitelist, JSON body parsers with payload limits and a /api/ping health-check endpoint.
- **Vault Cryptographic Engine (utils/crypto.js):** AES-256-GCM encryption and decryption using the native Node.js crypto module.
- **Authentication Middleware (middleware/auth.js):** issues and verifies JWTs carrying userId and role with 8-hour expiry.
- **RBAC Middleware (middleware/rbac.js):** guards that separate employee capabilities from admin privileges.
- **Validation Layer:** express-validator rules on authentication, credential creation and sharing endpoints.
- **Error Handler (middleware/errorHandler.js):** traps operational errors and returns standardised JSON without internal stack leakage.
- **Audit Logger (utils/auditLogger.js):** asynchronously records every sensitive action with client IP and timestamp.
- **Cascade Revocation Engine:** deletes all SharedAccess links where the deactivated user is owner or recipient.

4.2.3 Database and Support Modules

- **Mongoose Models:** User, Credential, SharedAccess and AuditLog.
- **Embedded Mongo Fallback:** mongodb-memory-server starts an in-memory database automatically when local MongoDB is not running and seeds the full demo dataset.

- **Database Seeder:** server/seed.js and database/seed.js populate realistic users (Admin Meera; Employees Aditya, Rahul, Sneha, Kiran), encrypted credentials with real IVs, shared access links and audit history.
- **Fixtures and Documentation:** database/sample_data.json and database/README.md document the schema and sample records.

4.3 DATABASE DESIGN

The system uses four normalised Mongoose collections. Their key fields, types and security details are given in Table 4.1.

Collection	Key Fields	Types & Constraints	Security & Indexing
users	name, email, masterPasswordHash, role, isActive, createdAt	email: String (unique, lowercase) masterPasswordHash: String (bcrypt) role: Enum (admin, employee) isActive: Boolean (default true)	Indexed by email. Master password hashed with bcrypt, 12 rounds. Inactive users are denied login and their shared access is revoked.
credentials	ownerId, title, username, url, encryptedPassword, iv, authTag, strengthScore, lastUpdated	ownerId: ObjectId (ref User) encryptedPassword: String (hex) iv: String (16 bytes hex) authTag: String (16 bytes hex) strengthScore: Number (0–4)	Indexed by ownerId. Passwords stored only as AES-256-GCM ciphertext; plaintext is never stored or logged.
sharedaccesses	credentialId, sharedWith, sharedBy, permission, createdAt	credentialId: ObjectId (ref Credential) sharedWith, sharedBy: ObjectId (ref User) permission: Enum (view, edit)	Compound index on credentialId + sharedWith. Deleting a credential or deactivating a user cascades deletion.
auditlogs	userId, action, credentialId, credentialTitle, details, ipAddress, timestamp	userId: ObjectId (ref User) action: Enum (login, create, view, edit, share, revoke, delete) ipAddress: String timestamp: Date	Indexed by timestamp and userId. Append-only, immutable; logs IP and action for forensic audits.

Table 4.1 Database Collections Schema Summary

4.4 CLASS DIAGRAM

The class diagram shows the four core entities and their relationships: a User owns many Credentials; a Credential can be shared through many SharedAccess records (each linking a sharedBy user and a sharedWith user); and a User generates many AuditLog entries that may refer to a Credential.

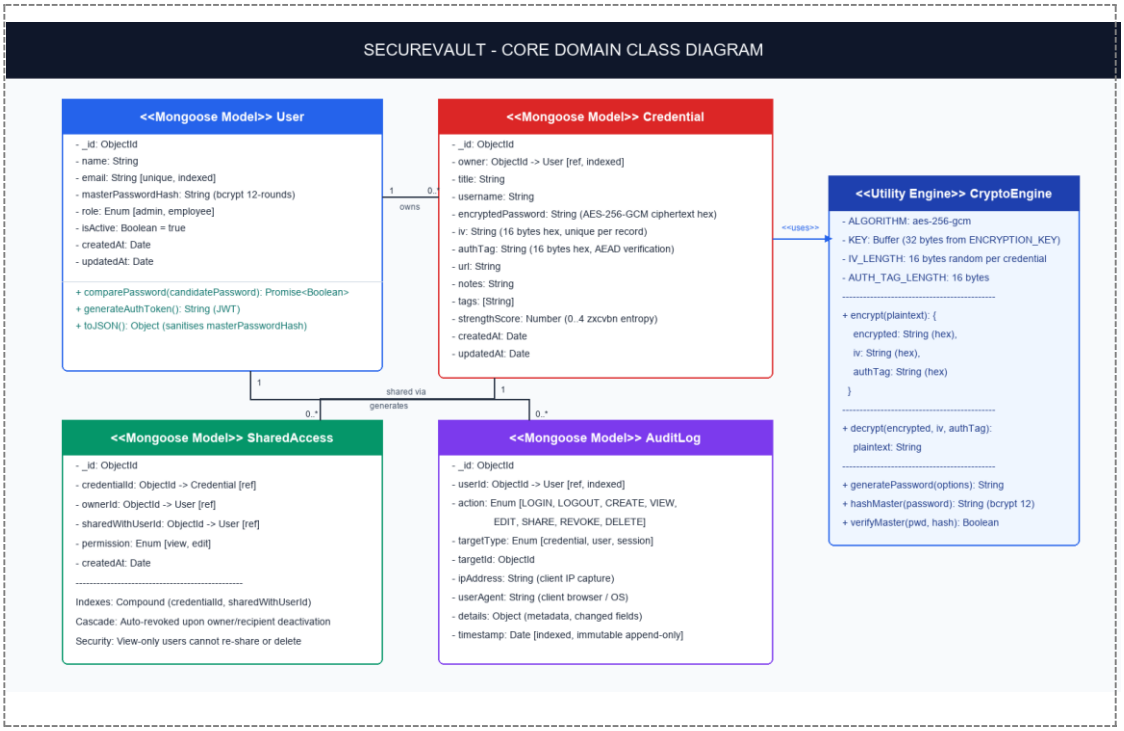


Figure 4.2 Class Diagram

4.5 SEQUENCE DIAGRAM

The sequence diagram illustrates the credential reveal flow: the user clicks Reveal in the React client; the Axios interceptor attaches the JWT; the server verifies the token, then verifies that the requester is the owner or holds an active SharedAccess grant; the cryptographic engine decrypts the ciphertext using the stored IV and authentication tag; the audit logger records the “view” action; and the plaintext is returned to the client.

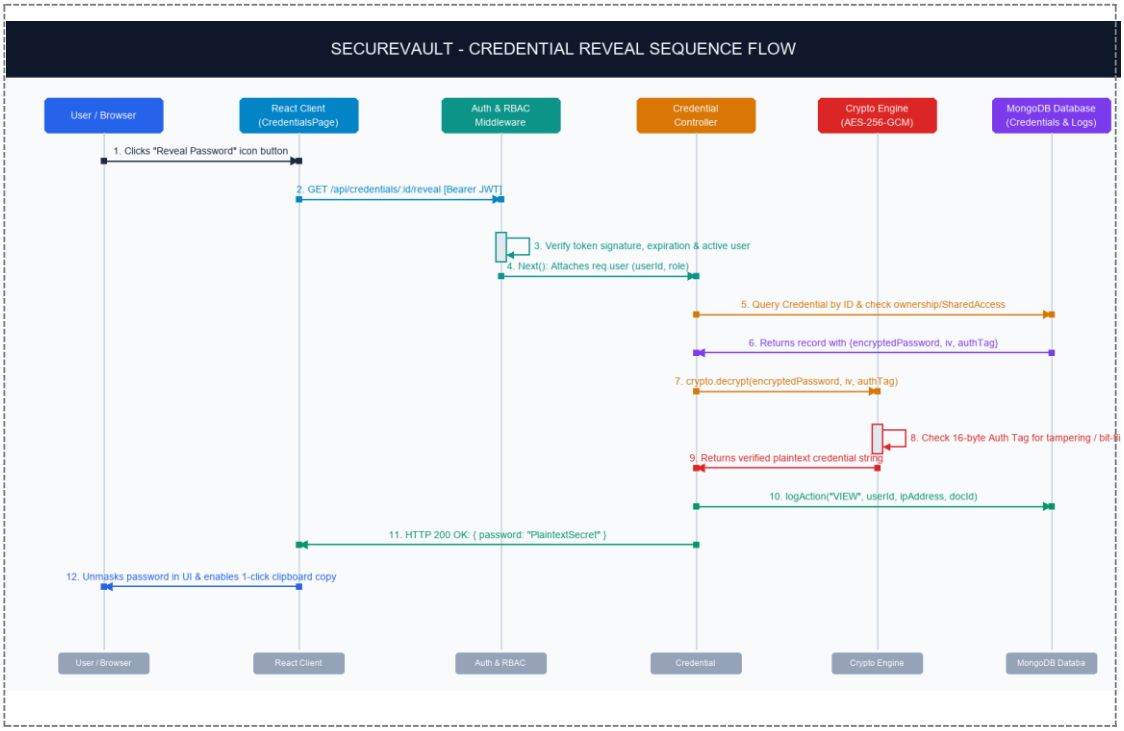


Figure 4.3 Sequence Diagram

4.6 KEY DESIGN DECISIONS

4.6.1 Hashing versus Encryption: Why Both Are Required

SecureVault deliberately uses both, for different responsibilities. **bcrypt hashing (one-way)** is used for master passwords, which are only ever used to authenticate identity. The system never needs to reconstruct the original; it only verifies a submitted password, so one-way hashing is the safest approach and even full database access does not let an attacker reverse it. **AES-256-GCM encryption (two-way)** is used for stored vault credentials such as AWS console passwords, database keys and server credentials, which authorised users must read back in order to log in to target systems. Hashes are irreversible, so reversible authenticated encryption is required.

4.6.2 AES-256-GCM instead of AES-CBC or Plain AES

AES-CBC provides confidentiality only and gives no guarantee of authenticity or integrity; an adversary who manipulates ciphertext bits can cause predictable plaintext changes (bit-flipping attacks). Galois/Counter Mode is an authenticated encryption with associated data (AEAD) mode that produces both ciphertext and a 16-byte authentication tag. During decryption Node.js checks the tag, and if even a single bit of the ciphertext or IV was altered in the database, decryption immediately fails. A

fresh 16-byte random IV per credential also ensures that identical passwords produce completely different ciphertexts.

4.6.3 Zero-Knowledge Administrative Boundary

System administrators are high-value targets, and if they could read employee credentials, an administrative compromise would compromise the whole enterprise. SecureVault therefore enforces strict decryption isolation: the /api/credentials/:id/reveal endpoint verifies that the requester is the owner or holds an active SharedAccess grant, so admins cannot decrypt employee credentials. Admin dashboards display only aggregate metrics (weak, reused and expired counts) and never query or receive ciphertext or plaintext values.

4.6.4 Stateless JWT Authentication versus Server Sessions

Traditional sessions require the server to maintain state in memory or in a centralised cache such as Redis, which adds synchronisation overhead and single points of failure in distributed deployments. JWTs are digitally signed and self-contained; the payload holds the user ID, role and expiry, and the server verifies the signature with its secret key without a database lookup on every route. This allows high-performance, stateless scaling across multiple backend instances.

CHAPTER 5

IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

SecureVault is implemented as a full-stack JavaScript application. The front end is a React 18 SPA that uses functional components and hooks (useState, useEffect, useContext, useMemo) and is built with Vite for fast start-up and Hot Module Replacement. The back end is an Express.js REST API organised in a modular Model-View-Controller structure with clear separation between routes, controllers, middleware, data models and utility services. Data is stored in MongoDB through Mongoose.

The technology choices and their justifications are summarised in Table 5.1.

Technology	**Role**	**Why It Was Used**
React.js (v18.3)	Frontend framework	Declarative, component-based UI with a Virtual DOM; reusable credential cards, strength meters, modals and tables; fast rendering without full page reloads.
Node.js (v24)	Runtime environment	Asynchronous, event-driven runtime; one language across front end and back end; native crypto module removes the need for third-party cipher libraries.
Express.js (v4.21)	Backend framework	Minimal, fast framework with a middleware pipeline so JWT, RBAC and validation run sequentially before controller logic.
MongoDB & Mongoose (v8.9)	Database & ODM	Document model fits credentials, IV/authTag metadata and audit trails; Mongoose adds schema validation, casting and ObjectId references.
Vite (v6.1)	Build tool	Native ES-module bundling, sub-second start-up, instant HMR and a built-in /api proxy that avoids CORS issues in development.
Node crypto (AES-256-GCM)	Authenticated encryption	Credentials must be retrievable, so reversible encryption is required; GCM adds a 16-byte authentication tag and a unique IV per record to prevent tampering and pattern attacks.

Technology	Role	Why It Was Used
bcrypt (v5.1)	Adaptive hashing	Master passwords are only verified, never shown; random salt and 12 work-factor rounds defeat brute-force and dictionary attacks even if the database leaks.
jsonwebtoken (v9.0)	Stateless authentication	Signed tokens carrying userId and role with 8-hour expiry; no server-side session store needed, allowing horizontal scaling.
zxcvbn (v4.4)	Strength evaluation	Pattern-based entropy estimation (dictionary words, names, sequences, l33t, keyboard walks) instead of naive regex rules.
dotenv (v16.4)	Environment isolation	Loads ENCRYPTION_KEY, JWT_SECRET and MONGO_URI from .env so secrets are never hard-coded or pushed to Git.

Table 5.1 Core Technologies and Rationale

Supporting libraries include express-validator (input validation and sanitisation against malformed data and NoSQL injection), cors (whitelisting the front-end origin http://localhost:3000), axios (interceptors for JWT headers and global 401/403 handling), react-router-dom (client-side routing and role-based route wrappers), lucide-react (consistent SVG icons), mongodb-memory-server (in-memory database fallback), nodemon (automatic server restart), concurrently (single-command start-up of the API and the Vite server with [BACKEND] and [FRONTEND] tags) and Postman (a test collection, postman_collection.json, that validates REST endpoints, token handoff and RBAC independently of the UI).

Project Organisation and Development Environment

- Root directory cleaned up: presentation decks in docs/presentations/, UI mockups in docs/screenshots/ and project summaries in docs/.
- A dedicated database/ folder with seed utilities, sample_data.json and a database architecture guide (database/README.md).
- A VS Code multi-root workspace file (SecureVault.code-workspace), tasks.json with “Start Full Stack”, “Start Backend Server”, “Start Frontend

Client" and "Seed Database", and launch.json for F5 debugging of Node.js controllers and seeders.

- The root package.json uses concurrently so that npm run dev starts the Express API and the Vite React server together.
- A production-grade .gitignore excludes node_modules, build artefacts, logs and server/.env; the Git repository was initialised on branch main with an initial commit, and a master README.md documents setup, demo credentials, database modes, API endpoints and GitHub push steps.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

5.2.1 AES-256-GCM Vault Engine (utils/crypto.js)

The engine uses the native Node.js crypto module. For each credential it generates a unique, cryptographically random 16-byte IV and encrypts the password with a 32-byte secret key. The resulting ciphertext, IV and 16-byte authentication tag are stored as hex strings. On decryption the tag is validated, and any tampering with the ciphertext or IV causes decryption to fail immediately. On update, the credential is re-encrypted with a new IV.

5.2.2 Master Password Hashing and Login

During registration the master password is hashed with bcrypt using 12 salt rounds. It is never stored in plaintext, never retrievable and never included in any response. At login, bcrypt compares the submitted password with the stored hash; inactive accounts are denied. On success the server issues a signed JWT (carrying userId and role) with an 8-hour expiry, and a "login" event is written to the audit log.

5.2.3 JWT Authentication and RBAC Middleware

middleware/auth.js verifies the token signature and expiry on every protected endpoint and attaches the user identity to the request. middleware/rbac.js then enforces strict separation between employee capabilities and admin management privileges. The same rules are mirrored on the client by the ProtectedRoute component, which checks the token, its expiry and the role before rendering a page.

5.2.4 Credential Creation, Reveal, Update and Delete

- **Create (POST /api/credentials):** validates input, computes a zxcvbn strength score, encrypts the password with AES-256-GCM and writes an audit log.
- **Read (GET /api/credentials):** returns owned and shared credential metadata only, never plaintext.
- **Reveal (GET /api/credentials/:id/reveal):** deciphers the password on demand after verifying that the requester is the owner or has an active share.
- **Update (PUT /api/credentials/:id):** allowed for the owner or an editor; re-encrypts with a new IV and logs the edit.
- **Delete (DELETE /api/credentials/:id):** owner only; cascades deletion of related SharedAccess links.

5.2.5 Sharing and Cascade Revocation

The owner can grant view or edit permission to a named colleague (POST /api/credentials/:id/share) and revoke it (DELETE /api/credentials/:id/share/:userId); both actions are audited. When an administrator deactivates an employee (PUT /api/admin/users/:id/status), the cascade revocation engine automatically deletes every SharedAccess record in which the user is either owner or recipient.

5.2.6 Password Health Analysis

The personal health endpoint (GET /api/health/my-health) calculates the vault security percentage and lists weak passwords, passwords reused across distinct domains and credentials unrotated for more than 90 days. The administrator endpoint (GET /api/admin/health) returns only the count of passwords in each strength band from Very Weak to Very Strong, so no plaintext is ever exposed.

5.2.7 Input Validation, Error Handling and Audit Logging

express-validator rules sanitise authentication, credential-creation and sharing requests (email format, password length thresholds, permission enums) and reject malformed requests before they reach controllers. The centralised error handler formats standard JSON error responses and prevents stack-trace leakage. The audit logger (utils/auditLogger.js) asynchronously records every view, create, edit, share, revoke and delete action with the client IP and timestamp, and the log is append-only.

5.2.8 Frontend Session Handling

AuthContext stores the user profile and JWT in localStorage and exposes login, registration and logout functions. The Axios instance in services/api.js adds the Bearer token to every request and, on a 401 response, redirects the user to the login page so that expired sessions end gracefully.

5.2.9 REST API Endpoints

Endpoint	HTTP	Access	Purpose & Action
/api/auth/register	POST	Public	Registers a new employee; validates input; hashes master password.
/api/auth/login	POST	Public	Verifies master password with bcrypt; issues signed 8-hour JWT.
/api/auth/profile	GET	Authenticated	Returns authenticated user profile (name, email, role).
/api/credentials	GET	Authenticated	Lists owned and shared credentials (metadata only).
/api/credentials	POST	Authenticated	Encrypts credential with AES-256-GCM; computes entropy score; writes audit log.
/api/credentials/:id/reveal	GET	Owner / Shared	Deciphers password on demand after ownership or share check.
/api/credentials/:id	PUT	Owner / Editor	Updates credential; re-encrypts with new IV; logs edit.
/api/credentials/:id	DELETE	Owner only	Deletes credential and cascades deletion of SharedAccess links.
/api/credentials/:id/share	POST	Owner only	Grants view or edit permission to a colleague; logs share.
/api/credentials/:id/share/:userId	DELETE	Owner only	Revokes colleague’s access; logs revoke.
/api/health/my-health	GET	Authenticated	Personal health percentage; weak, reused and old items.
/api/admin/dashboard	GET	Admin only	Organisation-wide counts, risk categories and recent audit activity.

Endpoint	HTTP	Access	Purpose & Action
/api/admin/users	GET	Admin only	Directory of employees, roles and active/inactive status.
/api/admin/users/:id/status	PUT	Admin only	Toggles active status; deactivation cascades revoke of all shared access.
/api/admin/health	GET	Admin only	Organisation-wide password strength distribution (0 to 4).
/api/admin/audit-logs	GET	Admin only	Tamper-evident audit logs with filters and CSV export.

Table 5.2 REST API Endpoint Specification

CHAPTER 6

CONCLUSION & FUTURE WORK

6.1 CONCLUSION

SecureVault delivers a comprehensive, enterprise-ready credential governance platform. By unifying modern software engineering practices (MERN stack, React 18, Vite, modular MVC, REST API) with defensive cryptographic standards (AES-256-GCM authenticated cipher, bcrypt salted hashing, zero-knowledge admin access, zxcvbn entropy scoring and immutable audit trails), the project demonstrates full-stack proficiency, strict data protection and real-world applicability.

The system removes both problems it set out to solve: weak and reused passwords are detected and discouraged through real-time strength feedback and health reports, and credential sprawl is replaced by encrypted storage, permission-controlled sharing with instant and cascading revocation, and a complete audit trail.

6.2 FUTURE ENHANCEMENTS

- Multi-factor authentication (for example TOTP) in addition to the master password.
- Encryption key rotation and support for an external key management service in place of a single environment key.
- Automated alerts or reminders when credentials become weak, reused or unrotated.
- A browser extension for auto-fill and secure capture of credentials.
- Deployment to a cloud platform with a managed MongoDB cluster and automated testing and CI/CD.

APPENDIX

Github Repository Link: https://github.com/Adity-kamath/SecureVault

SOURCE CODE:

1. Project Structure

```
SecureVault/
│
├── SecureVault.code-workspace
├── package.json            (root – "npm run dev" via concurrently)
├── README.md
├── .gitignore
├── .vscode/
│   ├── tasks.json
│   └── launch.json
├── server/
│   ├── middleware/
│   │   ├── auth.js
│   │   ├── rbac.js
│   │   └── errorHandler.js
│   ├── utils/
│   │   ├── crypto.js
│   │   └── auditLogger.js
│   ├── seed.js
│   ├── server.js
│   └── .env
│   (plus models, controllers, routes and db.js)
├── client/
│   └── src/
│       ├── services/api.js
│       ├── context/AuthContext
│       └── (components, pages, ProtectedRoute)
└── database/
    ├── seed.js
    ├── sample_data.json
    └── README.md
```

2. Backend Package Installation

```
npm install express mongoose bcrypt jsonwebtoken cors dotenv express-
validator zxcvbn
npm install --save-dev nodemon mongodb-memory-server
# root:  npm install --save-dev concurrently
# client: npm install react react-dom react-router-dom axios lucide-
react zxcvbn
```

3. Database Connection

```
// database/config/db.js - MongoDB Connection with MongoMemoryServer In-Memory
Fallback
const mongoose = require('mongoose');


const connectDB = async () => {
  const targetUri = process.env.MONGO_URI ||
'mongodb://127.0.0.1:27017/securevault';


  try {
    const conn = await mongoose.connect(targetUri, {
      serverSelectionTimeoutMS: 10000,
      autoIndex: true
    });
    console.log(`[Database] MongoDB Connected:
${conn.connection.host}/${conn.connection.name}`);


    // Auto-seed if the database is empty in production
    if (process.env.NODE_ENV === 'production') {
      const User = require('../models/User');
      const userCount = await User.countDocuments();
      if (userCount === 0) {
        console.log('[Database] Empty DB detected in production. Auto-seeding
demo data...');
        const seedData = require('../seed');
        await seedData();
      }
    }
  } catch (error) {
    console.warn(`[Database] Local MongoDB unavailable (${error.message}).
Starting embedded fallback...`);
    try {
      const { MongoMemoryServer } = require('mongodb-memory-server');
      const mongod = await MongoMemoryServer.create({ instance: { dbName:
'securevault' } });
      const memUri = mongod.getUri();


      if (mongoose.connection.readyState !== 0) {
        await mongoose.disconnect();
      }


      await mongoose.connect(memUri, { autoIndex: true });
      console.log(`[Database] Embedded MongoMemoryServer active at: ${memUri}`);


      // Auto-seed demo dataset (Admin Meera + Employees)
      const seedData = require('../seed');
      await seedData();
      console.log('[Database] Demo dataset seeded into in-memory database
successfully.');
    } catch (memErr) {
      console.error(`[Database Error] Embedded fallback failed:
${memErr.message}`);
    }
  }
};


module.exports = connectDB;
```

4. Environment Variables

```
PORT=5000
```

```
MONGO_URI=<your MongoDB connection string>
JWT_SECRET=<your secret>
ENCRYPTION_KEY=<32-byte key>
```

5. User Model

```
// database/models/User.js - Mongoose Schema with bcrypt Password Hashing &
Sanitization
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');


const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
      minlength: [2, 'Name must be at least 2 characters'],
      maxlength: [100, 'Name cannot exceed 100 characters']
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please provide a valid work email']
    },
    masterPasswordHash: {
      type: String,
      required: [true, 'Master password hash is required']
    },
    role: {
      type: String,
      enum: ['employee', 'admin'],
      default: 'employee'
    },
    isActive: {
      type: Boolean,
      default: true
    }
  },
  { timestamps: true }
);


// Method to verify candidate password against bcrypt master hash
userSchema.methods.comparePassword = async function (enteredPassword) {
  return bcrypt.compare(enteredPassword, this.masterPasswordHash);
};


// Static method to generate bcrypt hash with cost factor of 12 salt rounds
userSchema.statics.hashPassword = async function (plaintextPassword) {
  const salt = await bcrypt.genSalt(12);
  return bcrypt.hash(plaintextPassword, salt);
};


// Zero-knowledge sanitization: strip sensitive master hash from serialized JSON
userSchema.methods.toJSON = function () {
  const obj = this.toObject();
  delete obj.masterPasswordHash;
  delete obj.__v;
  return obj;
```

```
};

module.exports = mongoose.model('User', userSchema);
```

6. Password Hashing and Registration

```
// backend/controllers/authController.js - Registration Controller with zxcvbn
Strength Check
const register = async (req, res, next) => {
  try {
    const { name, email, masterPassword, role } = req.body;

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(409).json({
        success: false,
        message: 'An account with this email address already exists.'
      });
    }

    // Evaluate master password entropy using zxcvbn
    const health = evaluatePassword(masterPassword, [name, email]);
    if (health.score < 2) {
      return res.status(400).json({
        success: false,
        message: 'Master password is too weak. Please include symbols, mixed
case, and numbers.',
        feedback: health
      });
    }

    // Hash master password with 12 salt rounds using bcrypt
    const masterPasswordHash = await User.hashPassword(masterPassword);

    const user = await User.create({
      name,
      email,
      masterPasswordHash,
      role: role && ['employee', 'admin'].includes(role) ? role : 'employee',
      isActive: true
    });

    const token = generateToken(user);

    // Write immutable audit log entry
    await logAuditEvent({
      userId: user._id,
      action: 'register',
      req,
      details: `Account registered with role ${user.role}`
    });

    res.status(201).json({
      success: true,
      message: 'Account created successfully.',
      token,
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
```

```
        role: user.role,
        isActive: user.isActive,
        createdAt: user.createdAt
      }
    });
  } catch (error) {
    next(error);
  }
};
```

7. Login and JWT Generation

```
// backend/controllers/authController.js - User Authentication & JWT Generation
const generateToken = (user) => {
  return jwt.sign(
    {
      id: user._id,
      role: user.role,
      email: user.email
    },
    process.env.JWT_SECRET ||
'sv_jwt_sec_9f482a6e03c411ea8a60aa000400fe67ba8921dc8f7e2a9b',
    {
      expiresIn: process.env.JWT_EXPIRES_IN || '8h'
    }
  );
};


const login = async (req, res, next) => {
  try {
    const { email, masterPassword } = req.body;

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(401).json({
        success: false,
        message: 'Invalid email or master password.'
      });
    }

    if (!user.isActive) {
      return res.status(403).json({
        success: false,
        message: 'Your account has been deactivated by an administrator.'
      });
    }

    const isMatch = await user.comparePassword(masterPassword);
    if (!isMatch) {
      return res.status(401).json({
        success: false,
        message: 'Invalid email or master password.'
      });
    }

    const token = generateToken(user);

    // Record login in audit trail
    await logAuditEvent({
      userId: user._id,
      action: 'login',
```

```
      req,
      details: 'User authenticated successfully'
    });

    res.status(200).json({
      success: true,
      message: 'Authentication successful.',
      token,
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
        isActive: user.isActive,
        createdAt: user.createdAt
      }
    });
  } catch (error) {
    next(error);
  }
};
```

8. Authentication Middleware

```
// backend/middleware/auth.js - Stateless JWT Verification Middleware
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const authenticateToken = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({
        success: false,
        message: 'Access denied. No authorization token provided.'
      });
    }

    const token = authHeader.split(' ')[1];
    if (!token) {
      return res.status(401).json({
        success: false,
        message: 'Access denied. Malformed token.'
      });
    }

    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET ||
'sv_jwt_sec_9f482a6e03c411ea8a60aa000400fe67ba8921dc8f7e2a9b'
    );

    const user = await User.findById(decoded.id).select('-masterPasswordHash');
    if (!user) {
      return res.status(401).json({
        success: false,
        message: 'Session invalid. User account does not exist.'
      });
    }

    if (!user.isActive) {
```

```
      return res.status(403).json({
        success: false,
        message: 'Account has been deactivated. Please contact an administrator.'
      });
    }

    req.user = user;
    next();
  } catch (error) {
    if (error.name === 'TokenExpiredError') {
      return res.status(401).json({
        success: false,
        message: 'Session expired. Please log in again.'
      });
    }
    return res.status(401).json({
      success: false,
      message: 'Invalid or forged authentication token.'
    });
  }
};

module.exports = { authenticateToken };
```

9. Create API

```
// backend/controllers/credentialController.js - Encrypted Credential Creation
(AES-256-GCM)
const createCredential = async (req, res, next) => {
  try {
    const { title, username, password, url } = req.body;

    if (!password) {
      return res.status(400).json({
        success: false,
        message: 'Password is required to store credentials.'
      });
    }

    // 1. Evaluate password entropy with zxcvbn
    const health = evaluatePassword(password, [req.user.name, req.user.email,
title, username]);

    // 2. Authenticated encryption: generate random 16-byte IV, cipher text, and
16-byte GCM tag
    const { encryptedPassword, iv, authTag } = encryptPassword(password);

    // 3. Persist encrypted payload to database
    const credential = await Credential.create({
      ownerId: req.user._id,
      title: title.trim(),
      username: username.trim(),
      url: (url || '').trim(),
      encryptedPassword,
      iv,
      authTag,
      strengthScore: health.score,
      lastUpdated: new Date()
    });

    // 4. Record action in audit log
```

```
    await logAuditEvent({
      userId: req.user._id,
      action: 'create',
      credentialId: credential._id,
      credentialTitle: credential.title,
      req,
      details: `Created credential "${credential.title}" with strength score
${health.score}/4`
    });

    const responseObj = credential.toJSON();
    responseObj.accessLevel = 'owner';
    responseObj.isOwner = true;
    responseObj.sharedWith = [];

    res.status(201).json({
      success: true,
      message: 'Credential encrypted and saved successfully.',
      credential: responseObj
    });
  } catch (error) {
    next(error);
  }
};
```

10. Read API

```
// backend/controllers/credentialController.js - List Accessible Credentials &
On-Demand Reveal
// 1. Retrieve all credentials accessible to user (owned + shared records)
const getCredentials = async (req, res, next) => {
  try {
    const userId = req.user._id;

    const ownedCredentials = await Credential.find({ ownerId: userId })
      .sort({ updatedAt: -1 })
      .lean();

    const sharedAccessList = await SharedAccess.find({ sharedWith: userId })
      .populate({ path: 'credentialId', select: '-encryptedPassword -iv -authTag'
})
      .populate('sharedBy', 'name email')
      .lean();

    const formattedOwned = await Promise.all(
      ownedCredentials.map(async (cred) => {
        const shares = await SharedAccess.find({ credentialId: cred._id })
          .populate('sharedWith', 'name email')
          .lean();
        delete cred.encryptedPassword;
        delete cred.iv;
        delete cred.authTag;
        return {
          ...cred,
          accessLevel: 'owner',
          isOwner: true,
          sharedWith: shares.map((s) => ({
            userId: s.sharedWith?._id,
            name: s.sharedWith?.name,
            email: s.sharedWith?.email,
            permission: s.permission,
            sharedAt: s.createdAt
```

```
          }))
        };
      })
    );

    const formattedShared = sharedAccessList
      .filter((s) => s.credentialId != null)
      .map((s) => ({
        ...s.credentialId,
        accessLevel: s.permission,
        isOwner: false,
        sharedBy: { id: s.sharedBy?._id, name: s.sharedBy?.name, email:
s.sharedBy?.email },
        sharedAt: s.createdAt
      }));

    res.status(200).json({
      success: true,
      count: formattedOwned.length + formattedShared.length,
      credentials: [...formattedOwned, ...formattedShared]
    });
  } catch (error) {
    next(error);
  }
};

// 2. On-demand authenticated decryption & reveal
const revealPassword = async (req, res, next) => {
  try {
    const { id } = req.params;
    const { credential, accessLevel } = await resolveAccess(id, req.user._id);

    if (!credential || !accessLevel) {
      return res.status(404).json({
        success: false,
        message: 'Credential not found or you lack permission to view this
password.'
      });
    }

    const decryptedPassword = decryptPassword(credential.encryptedPassword,
credential.iv, credential.authTag);

    await logAuditEvent({
      userId: req.user._id,
      action: 'view',
      credentialId: credential._id,
      credentialTitle: credential.title,
      req,
      details: `Revealed password for "${credential.title}" (Access:
${accessLevel})`
    });

    res.status(200).json({
      success: true,
      password: decryptedPassword,
      strengthScore: credential.strengthScore
    });
  } catch (error) {
    next(error);
```

```
  }
};
```

11. Update

```
// backend/controllers/credentialController.js - Update Credential with Re-
encryption
const updateCredential = async (req, res, next) => {
  try {
    const { id } = req.params;
    const { title, username, password, url } = req.body;

    const { credential, accessLevel } = await resolveAccess(id, req.user._id);
    if (!credential) {
      return res.status(404).json({ success: false, message: 'Credential not
found' });
    }

    if (accessLevel !== 'owner' && accessLevel !== 'edit') {
      return res.status(403).json({
        success: false,
        message: 'Forbidden: You have view-only access to this credential.'
      });
    }

    if (title !== undefined) credential.title = title.trim();
    if (username !== undefined) credential.username = username.trim();
    if (url !== undefined) credential.url = url.trim();

    let passwordUpdated = false;
    if (password) {
      const health = evaluatePassword(password, [req.user.name, req.user.email,
credential.title, credential.username]);
      const enc = encryptPassword(password);
      credential.encryptedPassword = enc.encryptedPassword;
      credential.iv = enc.iv;
      credential.authTag = enc.authTag;
      credential.strengthScore = health.score;
      passwordUpdated = true;
    }

    credential.lastUpdated = new Date();
    await credential.save();

    await logAuditEvent({
      userId: req.user._id,
      action: 'edit',
      credentialId: credential._id,
      credentialTitle: credential.title,
      req,
      details: `Updated credential "${credential.title}". Password updated:
${passwordUpdated}`
    });

    const responseObj = credential.toJSON();
    responseObj.accessLevel = accessLevel;
    responseObj.isOwner = accessLevel === 'owner';

    res.status(200).json({
      success: true,
      message: 'Credential updated successfully.',
```

```
      credential: responseObj
    });
  } catch (error) {
    next(error);
  }
};
```

12. Delete

```
// backend/controllers/credentialController.js - Delete Credential & Cascade
Revoke Shares
const deleteCredential = async (req, res, next) => {
  try {
    const { id } = req.params;
    const credential = await Credential.findById(id);


    if (!credential) {
      return res.status(404).json({ success: false, message: 'Credential not
found.' });
    }


    // Strict ownership boundary: only owner can delete credential
    if (credential.ownerId.toString() !== req.user._id.toString()) {
      return res.status(403).json({
        success: false,
        message: 'Forbidden: Only the credential owner can delete it.'
      });
    }


    // Cascade deletion: purge all SharedAccess entries referencing this
credential
    const sharesRevoked = await SharedAccess.deleteMany({ credentialId:
credential._id });


    await Credential.findByIdAndDelete(id);


    await logAuditEvent({
      userId: req.user._id,
      action: 'delete',
      credentialId: credential._id,
      credentialTitle: credential.title,
      req,
      details: `Deleted credential "${credential.title}". Purged
${sharesRevoked.deletedCount} shared grants.`
    });


    res.status(200).json({
      success: true,
      message: 'Credential and all associated shared access records deleted
successfully.'
    });
  } catch (error) {
    next(error);
  }
};
```

13. Express Routes

```
// backend/routes/credentialRoutes.js - REST API Route Definitions & Middleware
Pipeline
const express = require('express');
const router = express.Router();
```

```
const {
  getCredentials,
  createCredential,
  getCredentialById,
  revealPassword,
  updateCredential,
  deleteCredential,
  generatePassword
} = require('../controllers/credentialController');
const {
  shareCredential,
  revokeShare,
  getCredentialShares,
  getActiveColleagues
} = require('../controllers/shareController');
const { authenticateToken } = require('../middleware/auth');
const {
  credentialValidation,
  credentialUpdateValidation,
  shareValidation
} = require('../middleware/validate');

// All credential routes require active session JWT
router.use(authenticateToken);

// Active colleagues lookup for sharing selector
router.get('/colleagues/active', getActiveColleagues);

// List and Create
router.get('/', getCredentials);
router.post('/', credentialValidation, createCredential);

// Cryptographic password generator
router.post('/generate', generatePassword);

// Colleague sharing endpoints
router.get('/:id/share', getCredentialShares);
router.post('/:id/share', shareValidation, shareCredential);
router.delete('/:id/share/:userId', revokeShare);

// CRUD operations on individual credential
router.get('/:id', getCredentialById);
router.get('/:id/reveal', revealPassword);
router.put('/:id', credentialUpdateValidation, updateCredential);
router.delete('/:id', deleteCredential);

module.exports = router;
```

14. Main Server File

```
// backend/server.js - Express App Bootstrap, Middleware Stack & Server Startup
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('../database/config/db');
const errorHandler = require('./middleware/errorHandler');

// Initialize database connection (with MongoMemoryServer development fallback)
connectDB();
```

```
const app = express();

// Security and request parsing
app.use(cors({ origin: process.env.CLIENT_URL || '*', credentials: true }));
app.use(express.json({ limit: '50kb' }));
app.use(express.urlencoded({ extended: true }));

// Liveness probe
app.get('/api/ping', (req, res) => {
  res.status(200).json({ status: 'online', timestamp: new Date().toISOString(),
service: 'SecureVault REST API' });
});

// Mount modular API routers
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/credentials', require('./routes/credentialRoutes'));
app.use('/api/health', require('./routes/healthRoutes'));
app.use('/api/admin', require('./routes/adminRoutes'));

// Production client serving
const path = require('path');
if (process.env.NODE_ENV === 'production') {
  app.use(express.static(path.join(__dirname, '../frontend/dist')));
  app.get('*', (req, res, next) => {
    if (req.path.startsWith('/api')) return next();
    res.sendFile(path.resolve(__dirname, '../frontend/dist', 'index.html'));
  });
}

// 404 Catch-all handler
app.use((req, res, next) => {
  res.status(404).json({ success: false, message: `Route not found: ${req.method}
${req.originalUrl}` });
});

// Centralized error handler
app.use(errorHandler);

const PORT = process.env.PORT || 5000;
if (process.env.NODE_ENV !== 'test') {
  app.listen(PORT, () => {
    console.log(`[SecureVault API] Server running on port ${PORT} in
${process.env.NODE_ENV || 'development'} mode.`);
  });
}

module.exports = app;
```

15. React Login Component

```
// frontend/src/pages/LoginPage.jsx - React 18 Login Screen with Quick-Fill Demo
Accounts
import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { Shield, Lock, Mail, ArrowRight, UserCheck } from 'lucide-react';

const LoginPage = () => {
  const [email, setEmail] = useState('');
  const [masterPassword, setMasterPassword] = useState('');
```

```
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    try {
      const res = await login(email, masterPassword);
      if (res.user?.role === 'admin') {
        navigate('/admin/dashboard');
      } else {
        navigate('/credentials');
      }
    } catch (err) {
      setError(err.response?.data?.message || 'Login failed. Please verify your
credentials.');
    } finally {
      setLoading(false);
    }
  };

  const handleDemoFill = (demoEmail, role) => {
    setEmail(demoEmail);
    setMasterPassword('MasterPassword123!@#');
  };

  return (
    <div style={{ minHeight: '100vh', display: 'flex', alignItems: 'center',
justifyContent: 'center', background: '#f4f7f5', padding: '20px' }}>
      <div className="card" style={{ maxWidth: '440px', width: '100%', padding:
'36px' }}>
        <div style={{ textAlign: 'center', marginBottom: '28px' }}>
          <div className="brand-icon" style={{ margin: '0 auto 16px auto', width:
'48px', height: '48px' }}>
            <Shield size={28} />
          </div>
          <h1 style={{ fontSize: '24px', fontWeight: 700, color: '#0f172a'
}}>Sign in to SecureVault</h1>
          <p style={{ fontSize: '13px', color: '#64748b', marginTop: '6px'
}}>Zero-knowledge encrypted credential management</p>
        </div>

        {error && (
          <div style={{ background: '#fee2e2', color: '#b91c1c', padding: '10px
14px', borderRadius: '8px', marginBottom: '18px', fontSize: '13px' }}>
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit}>
          <div className="form-group">
            <label className="form-label">Work Email</label>
            <input type="email" placeholder="name@company.com" className="form-
input" value={email} onChange={(e) => setEmail(e.target.value)} required />
          </div>
```

```
          <div className="form-group">
            <label className="form-label">Master Password</label>
            <input type="password" placeholder="••••••••••••••••"
className="form-input" value={masterPassword} onChange={(e) =>
setMasterPassword(e.target.value)} required />
          </div>

          <button type="submit" disabled={loading} className="btn btn-primary"
style={{ width: '100%', marginTop: '8px' }}>
            <span>{loading ? 'Authenticating...' : 'Sign In'}</span>
            <ArrowRight size={16} />
          </button>
        </form>

        <div style={{ marginTop: '24px', paddingTop: '20px', borderTop: '1px
solid #f1f5f9' }}>
          <span style={{ fontSize: '11px', textTransform: 'uppercase',
letterSpacing: '0.05em', color: '#94a3b8', fontWeight: 700, display: 'block',
marginBottom: '10px' }}>
            Instant Demo Logins
          </span>
          <div style={{ display: 'flex', gap: '8px' }}>
            <button type="button" onClick={() =>
handleDemoFill('meera@company.com', 'admin')} className="btn btn-secondary btn-
sm" style={{ flex: 1, fontSize: '12px' }}>
              <UserCheck size={13} color="#0f766e" />
              <span>Admin (Meera)</span>
            </button>
            <button type="button" onClick={() =>
handleDemoFill('aditya@company.com', 'employee')} className="btn btn-secondary
btn-sm" style={{ flex: 1, fontSize: '12px' }}>
              <UserCheck size={13} color="#059669" />
              <span>Employee (Aditya)</span>
            </button>
          </div>
        </div>
      </div>
    </div>
  );
};

export default LoginPage;
```

16. Admin Dashboard Logic

```
// frontend/src/pages/AdminDashboardPage.jsx - Admin Security Governance Data
Fetching
import React, { useState, useEffect } from 'react';
import { adminService } from '../services/adminService';

const AdminDashboardPage = () => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchDashboard = async () => {
      try {
        setLoading(true);
        // Calls GET /api/admin/dashboard (Zero-knowledge: aggregated metadata
only)
        const res = await adminService.getDashboard();
```

```
        if (res.success) {
          setData(res.dashboard);
        }
      } catch (err) {
        console.error('Failed to load admin dashboard:', err);
      } finally {
        setLoading(false);
      }
    };

    fetchDashboard();
  }, []);

  if (loading) {
    return <div style={{ textAlign: 'center', padding: '60px', color: '#64748b'
}}>Loading organisation security overview...</div>;
  }

  const { stats, issues, recentActivity } = data || { stats: {}, issues: {},
recentActivity: [] };
  // stats contains: { employees: 3, credentials: 12, orgHealthScore: 84,
needAttention: 2 }
  // issues contains: { weak: 1, reused: 2, old: 1 }
  // recentActivity contains the 10 most recent immutable audit log records

  return (
    <div>
      <div className="page-header">
        <h1 className="page-title">Organisation Overview</h1>
        <p className="page-subtitle">Acme Technologies · live security posture
across all employees</p>
      </div>
      {/* Metrics Cards, Issue Ratios, and Real-Time Audit Activity Feed rendered
here */}
    </div>
  );
};

export default AdminDashboardPage;
```

17. REST API Request and Response Examples

```
/* 17. REST API Request and Response Examples */

// Example 1: User Login
// POST /api/auth/login
// Request Body:
{
  "email": "aditya@company.com",
  "masterPassword": "MasterPassword123!@#"
}

// Response (HTTP 200 OK):
{
  "success": true,
  "message": "Authentication successful.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "660c1f20b3a8c512d4e80001",
    "name": "Aditya K.",
    "email": "aditya@company.com",
```

```
    "role": "employee",
    "isActive": true,
    "createdAt": "2026-03-15T09:30:00.000Z"
  }
}

// Example 2: Create Encrypted Credential
// POST /api/credentials
// Headers: Authorization: Bearer <token>
// Request Body:
{
  "title": "AWS Production Console",
  "username": "sysadmin@company.com",
  "password": "V!8x#kP9$2mQ@wZ7",
  "url": "https://signin.aws.amazon.com/console"
}

// Response (HTTP 201 Created):
{
  "success": true,
  "message": "Credential encrypted and saved successfully.",
  "credential": {
    "id": "660c2b50b3a8c512d4e80042",
    "title": "AWS Production Console",
    "username": "sysadmin@company.com",
    "url": "https://signin.aws.amazon.com/console",
    "strengthScore": 4,
    "accessLevel": "owner",
    "isOwner": true,
    "sharedWith": [],
    "createdAt": "2026-03-20T11:15:30.000Z",
    "updatedAt": "2026-03-20T11:15:30.000Z"
  }
}

// Example 3: Reveal Credential Password On-Demand
// GET /api/credentials/660c2b50b3a8c512d4e80042/reveal
// Headers: Authorization: Bearer <token>

// Response (HTTP 200 OK):
{
  "success": true,
  "password": "V!8x#kP9$2mQ@wZ7",
  "strengthScore": 4
}
```

SCREENSHOTS OF THE PROJECT

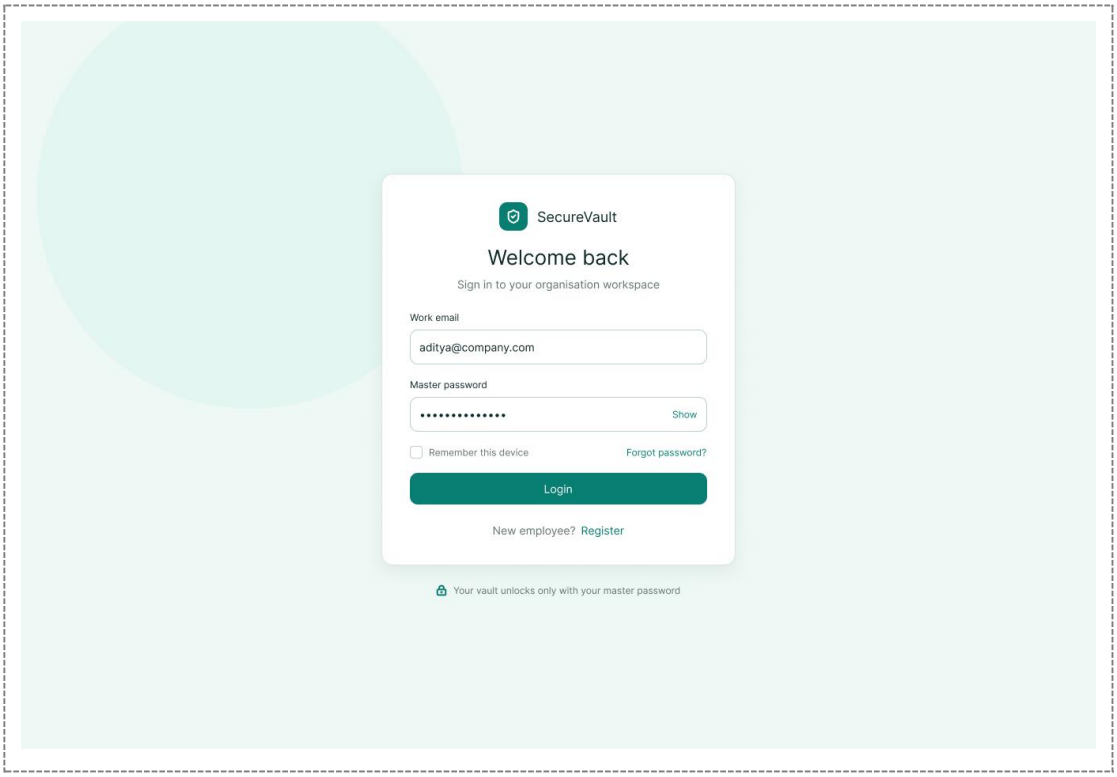


Figure A.1 Login screen with Demo Account quick-fill buttons

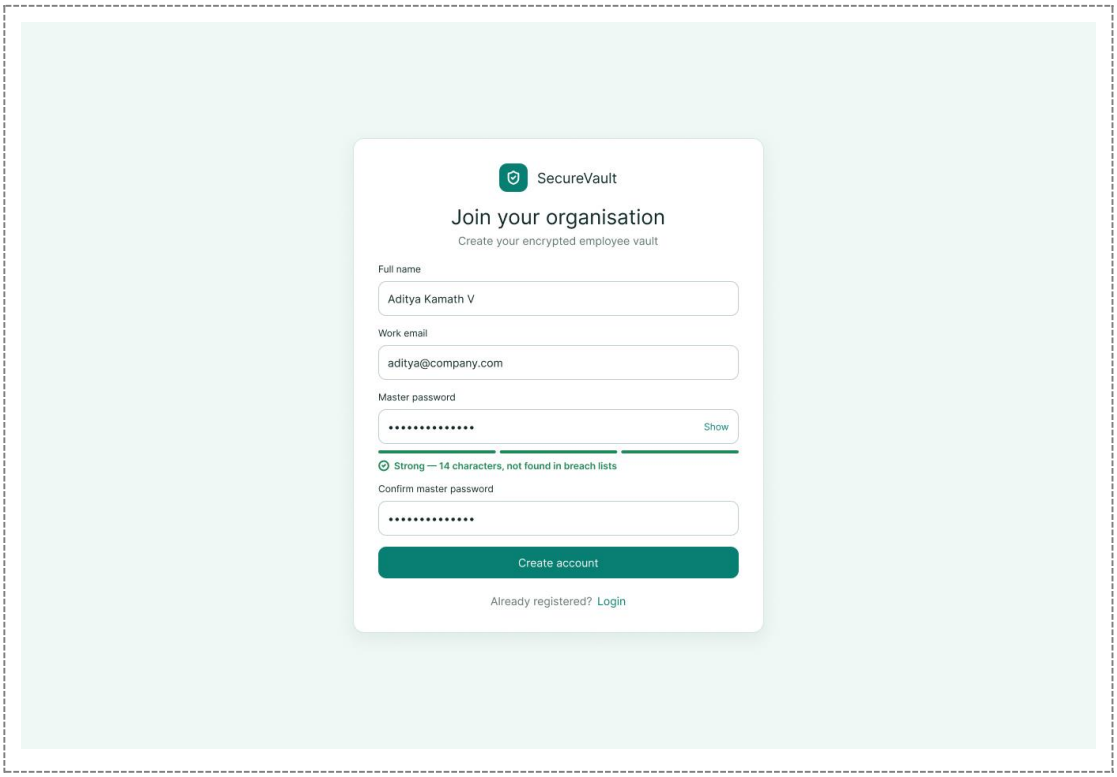


Figure A.2 Registration screen with real-time zxcvbn strength meter

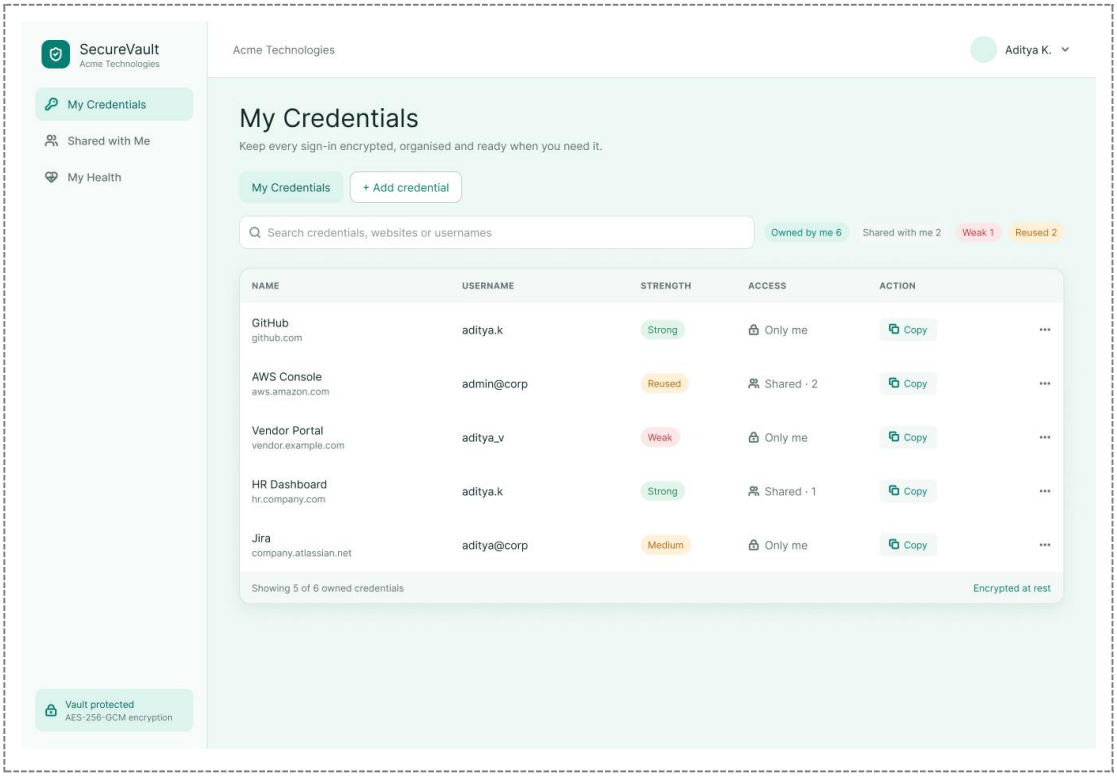


Figure A.3 Personal credentials vault (search, filters, owned/shared badges)

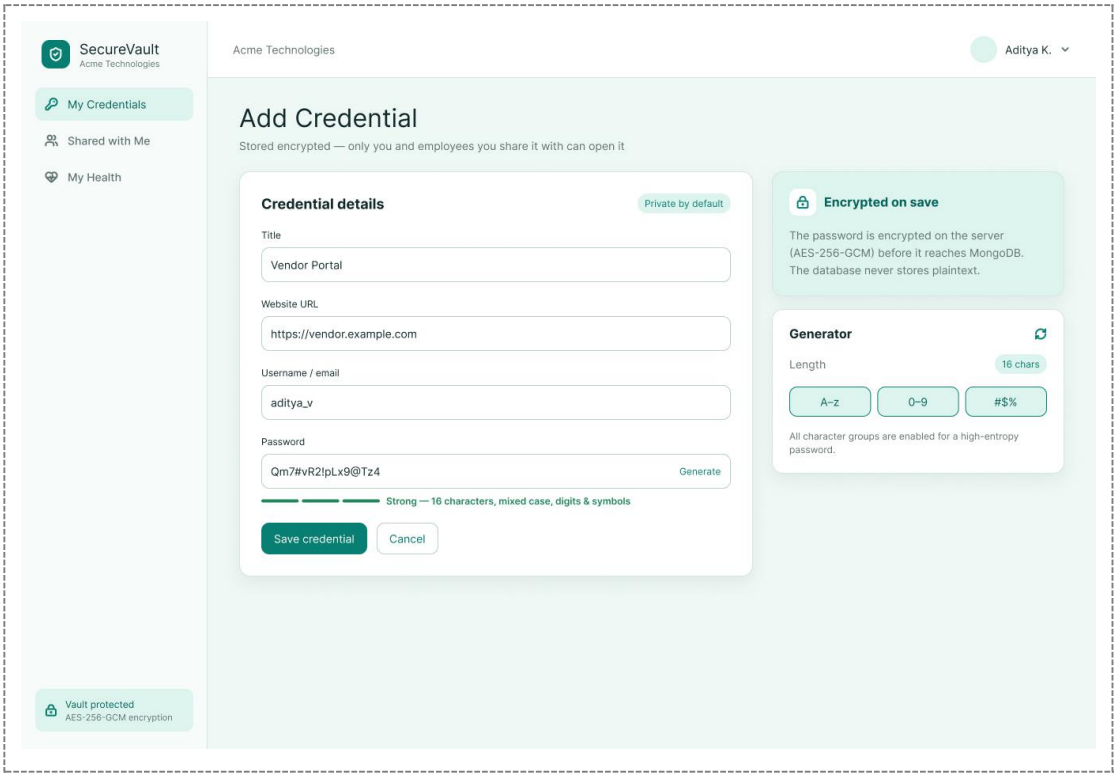


Figure A.4 Add/Edit credential modal with password generator

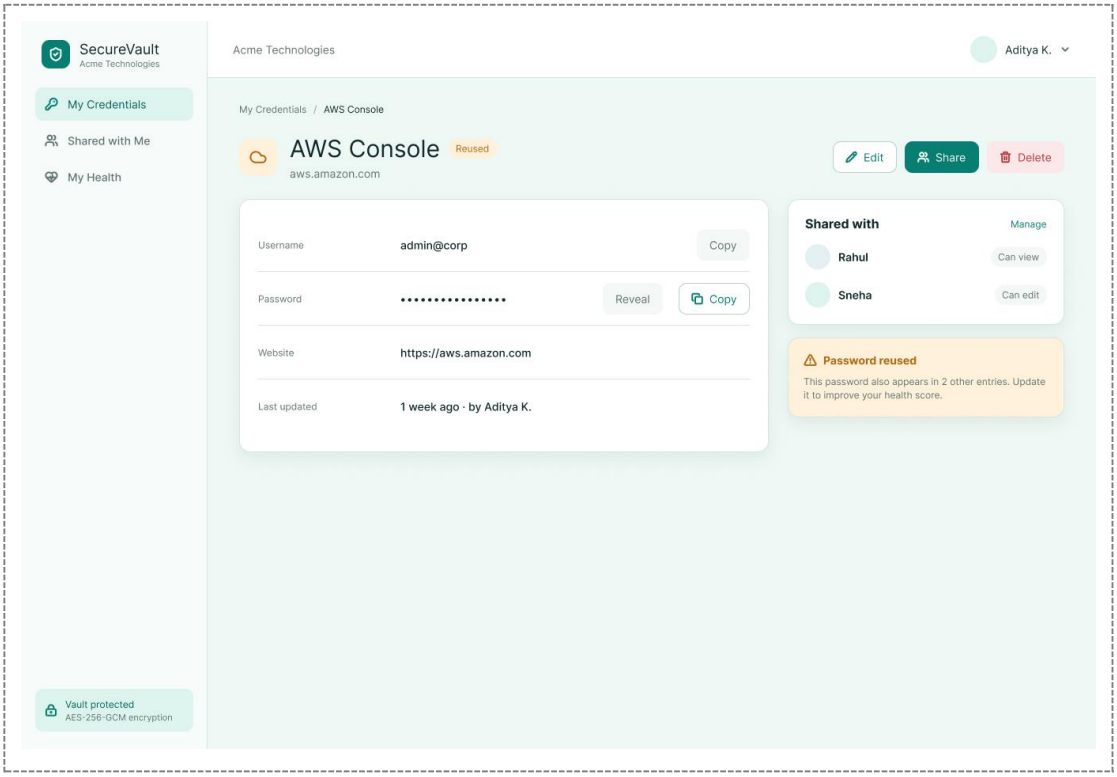


Figure A.5 Credential reveal and copy

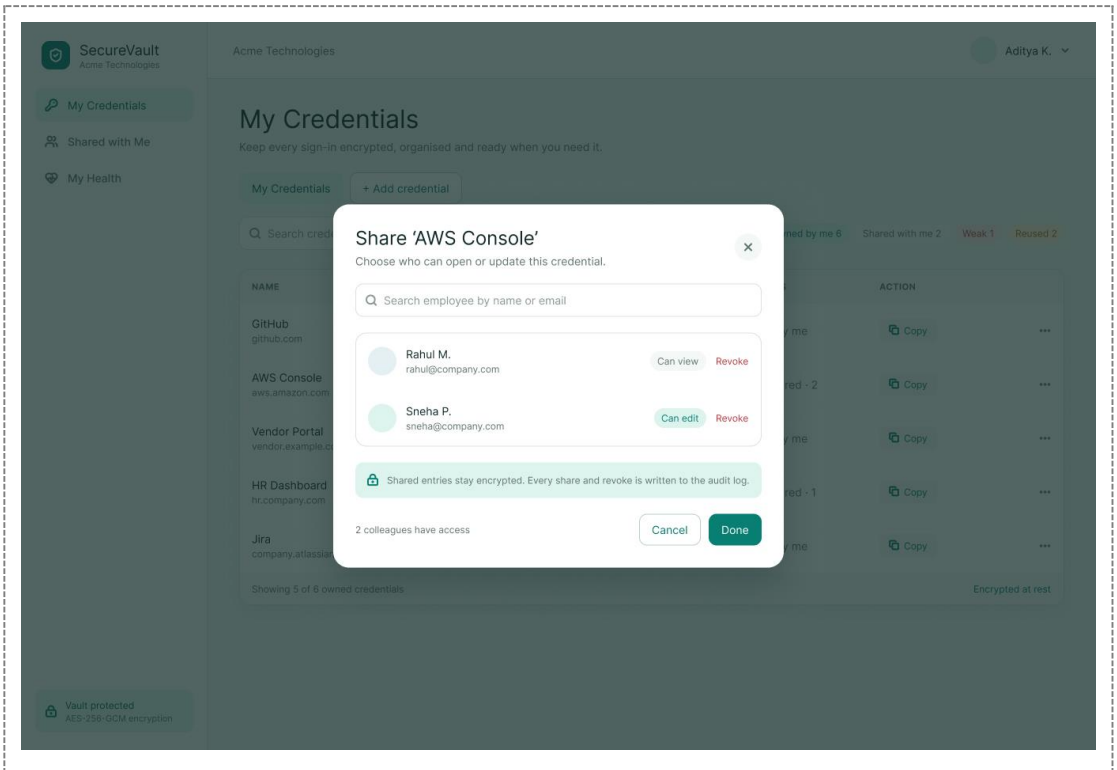


Figure A.6 Colleague sharing modal

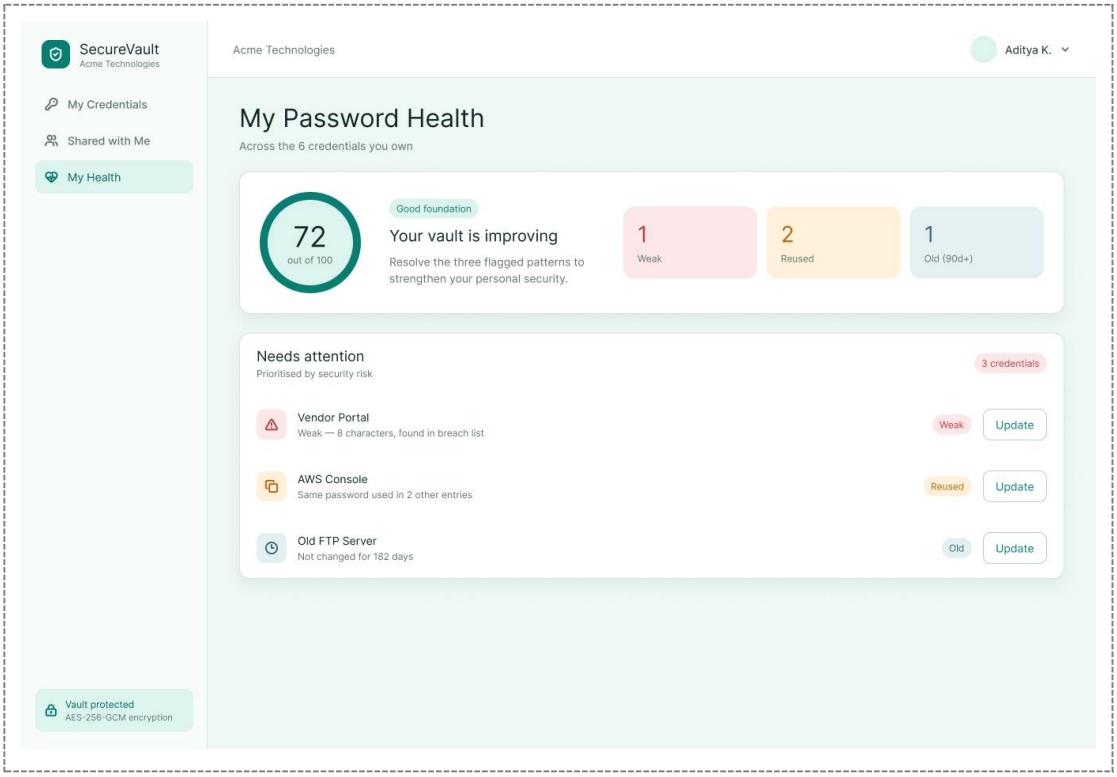


Figure A.7 Personal password health report

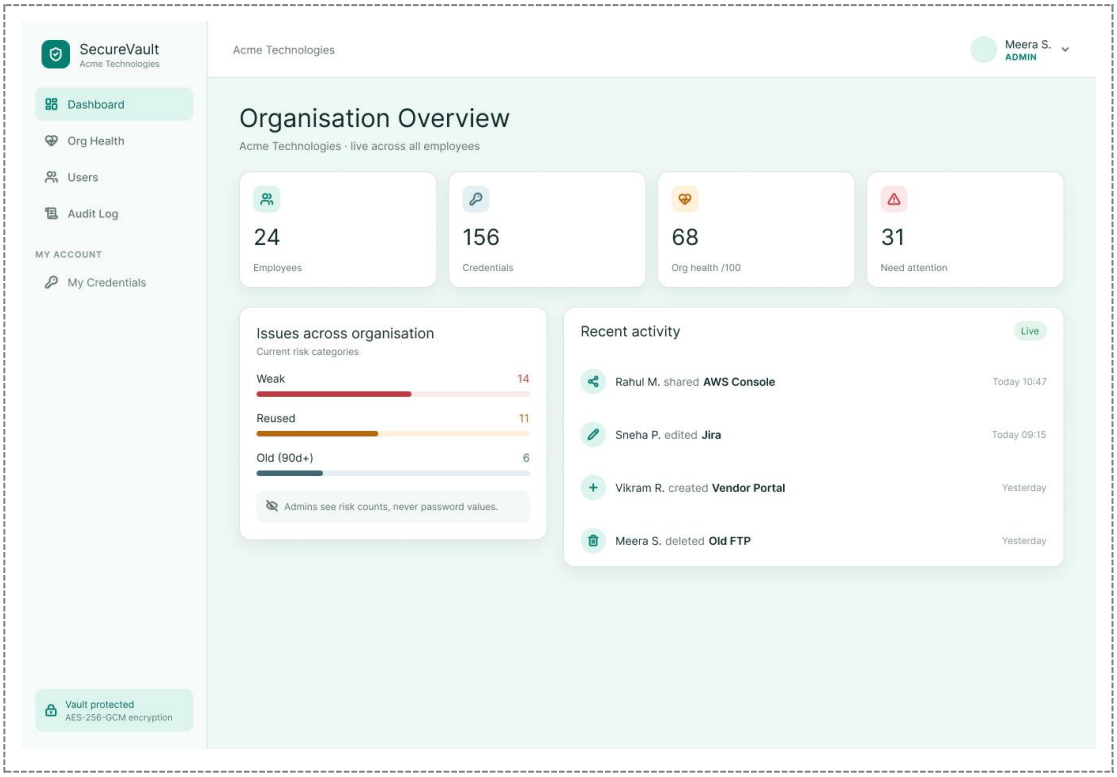


Figure A.8 Admin governance dashboard

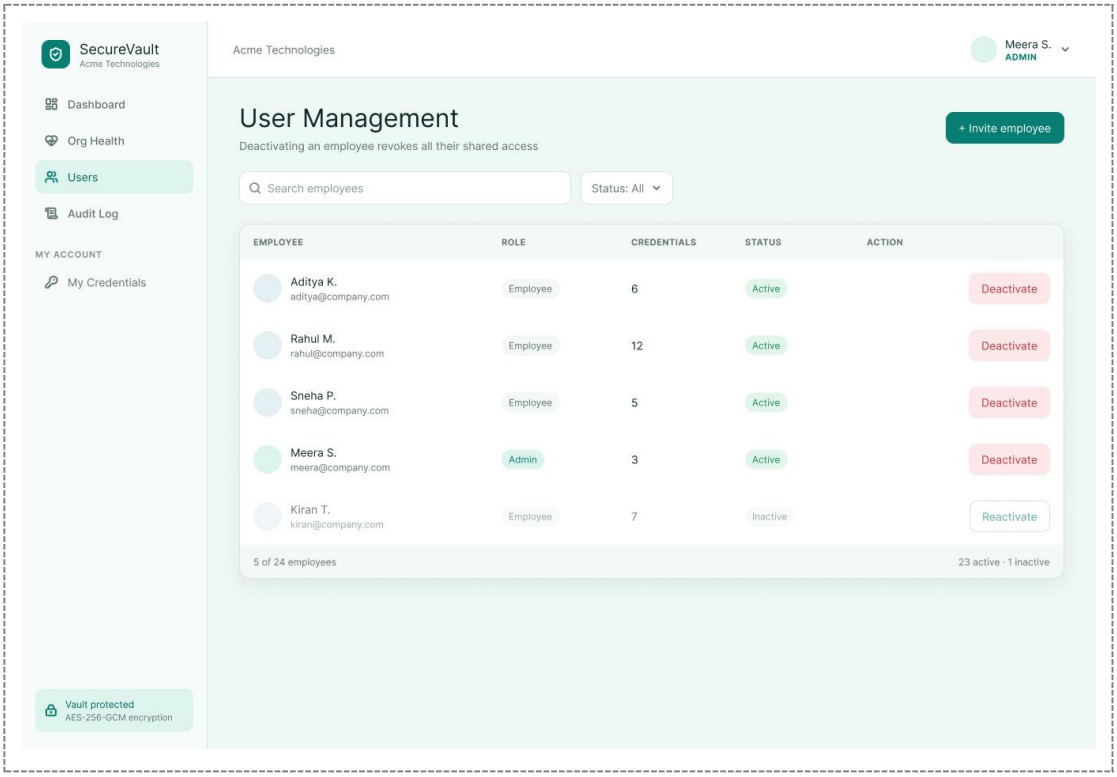


Figure A.9 Admin user management

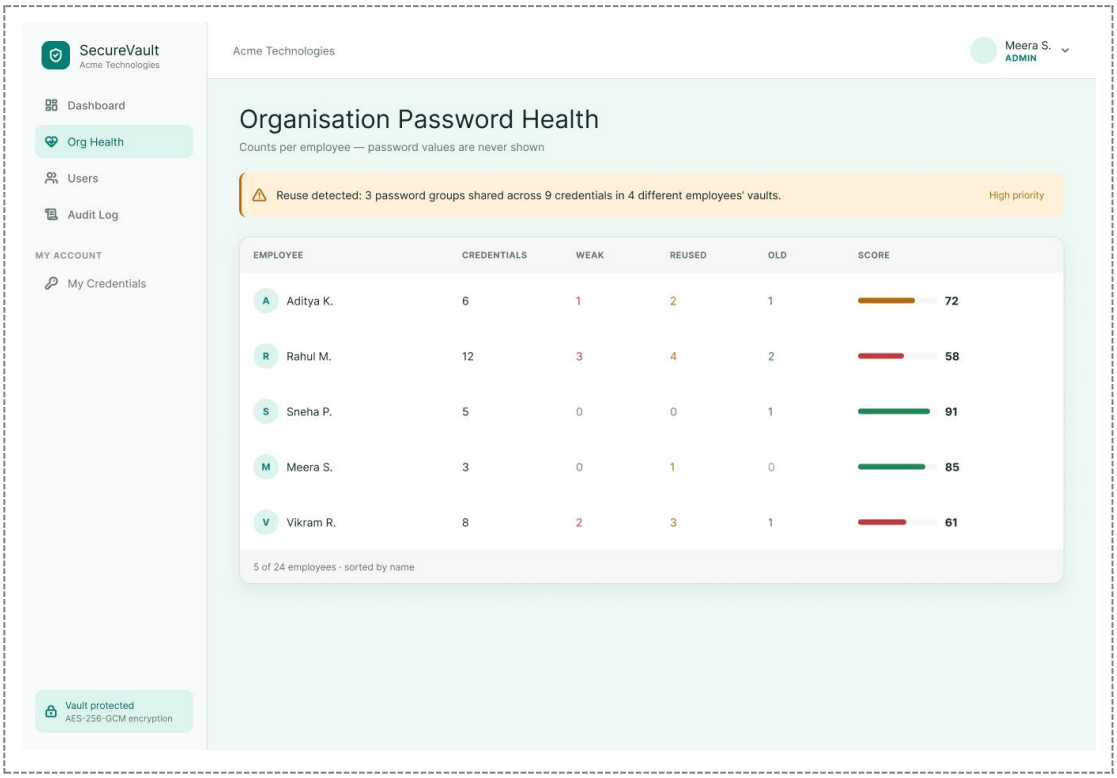


Figure A.10 Organisation password health distribution

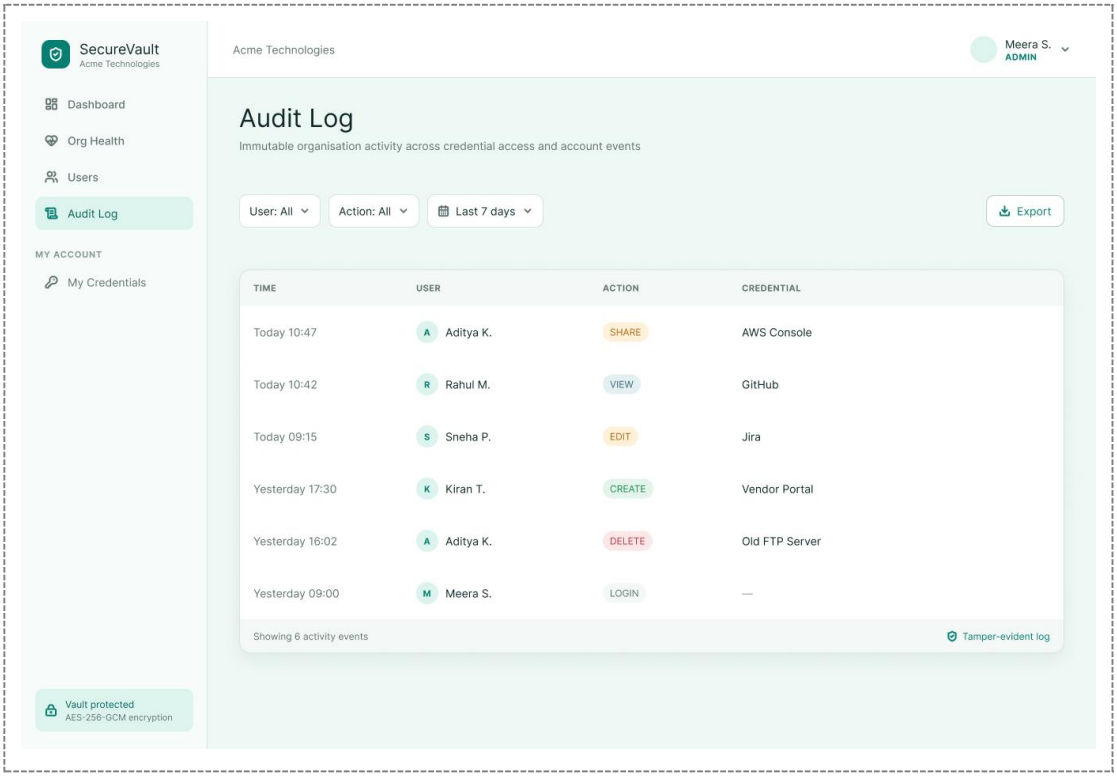


Figure A.11 Admin audit log with CSV export

REFERENCES

[1] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in Proc. USENIX Annual Technical Conference (FREENIX Track), 1999.

[2] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST Special Publication 800-38D, 2007.

[3] D. L. Wheeler, “zxcvbn: Low-Budget Password Strength Estimation,” in Proc. 25th USENIX Security Symposium, 2016.

[4] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.

[5] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series.

[6] Node.js Documentation – Crypto module. https://nodejs.org/api/crypto.html

[7] React Documentation. https://react.dev

[8] Express.js Documentation. https://expressjs.com

[9] MongoDB Documentation. https://www.mongodb.com/docs

[10] Mongoose Documentation. https://mongoosejs.com

[11] Vite Documentation. https://vitejs.dev